Day 9 – Laravel Advanced Migrations

45 min Concept • 90 min Coding • 30 min Practice • 15 min Revision

1. Migration – Quick Recap

Migration ■■■■■■ database table structure-■ PHP code ■■■■■ define ■■■■■■, ■■■ Git-■■ track ■■■■■ ■■■■■■ Laravel feature. Migration file-■■ **up()** method change-■ apply ■■■■■■■■; **down()** method ■■■ reverse ■■■■■■■■.

> **Important:** Migration file ■■■■■■■■ ■■■■■■■ database-■■ table ■■■■■■■■■■■. **php artisan migrate** run ■■■■■ ■■■■■■■■.

2. Foreign Key (FK)

Foreign key ■■■■■■ ■■■ table-■■ ■■■■■■■■■ column, ■■■■■■■ table-■■ record-■ reference ■■■■■■■■■■■ database constraint. Example: **enrollments.course_id** → **courses.id**.

```
Laravel:
$table->foreignId('course_id')->constrained('courses');
```

constrained() relationship-■ define ■■■■■■■■■. Convention-■ follow ■■■■■■■■ **foreignId('course_id')->constrained()** ■■■■■ short form ■■■■■■■■■■■■■■■.

3. Foreign Key – Delete Behaviour

cascadeOnDelete() – parent record delete ■■■■■ related child records-■■■■ delete ■■■■■■.

```
$table->foreignId('course_id')->constrained()->cascadeOnDelete();
```

restrictOnDelete() – related records ■■■■■■■■■ ■■■■■ parent record delete ■■■■■■ database prevent ■■■■■■■■■.

```
$table->foreignId('course_id')->constrained()->restrictOnDelete();
```

nullOnDelete() – parent delete ■■■■■■ FK value NULL ■■■■■■; column nullable ■■ ■■■■■■■ ■■■■■■■■■.

```
$table->foreignId('course_id')->nullable()->constrained()->nullOnDelete();
```

4. unique()

unique() ■■■■ column-■■ duplicate value save ■■■■■■■ database-level rule ■■■■■■■■■■■■.

```
$table->string('name')->unique();
```

■■■■ student ■■■■ course-■■ duplicate enrollment ■■■■■■■■■■■■■■■ ■■■■■■■■ ■■■■■■■ columns-■■ combination-■ unique ■■■■■■■■■■:

```
$table->unique(['student_id', 'course_id']);
```

■■■■■■■ (student_id=1, course_id=1) ■■■■■■■■■■ ■■■■■ insert ■■■■■■ ■■■■■■■■■. ■■■■■■ (1,2) ■■■■■■■ (2,1) allowed.

5. index()

Index ■■■■■■■ database query/search performance-■ improve ■■■■■■ ■■■■■■■ structure. ■■■■■■■■ search/filter ■■■■■■■■■■■■■ columns-■■ index useful ■■ ■■■■■■■■■■■■.

```
$table->index('name');
```

Difference: Foreign key = relationship/integrity; Unique = duplicate prevention; Index = query performance. Unique constraint ■■■■■■■■ unique index-■■■■■■ ■■■■■■■■■■■■.

6. nullable()

Column-■■ value ■■■■■■■■■ **NULL** ■■■■■■■■■■■■ ■■■■■■■■ **nullable()** ■■■■■■■■■■■■■■■■.

```
$table->text('description')->nullable();
```

NULL ■■■■■■■■ empty string ('') ■■■■■■■■. NULL = value ■■■■■■/unknown; empty string = value empty.

7. default()

Insert ■■■■■■■■■■■■■ value ■■■■■■■■■■■■■■■■■■■ ■■■■■■■■ database ■■■■■■■■■■■ ■■■■■■■■ default value-■ define ■■■■■■ **default()** ■■■■■■■■■■■■■■■■.

```
$table->boolean('is_active')->default(true);
$table->string('status')->default('pending');
$table->decimal('price', 8, 2)->default(0);
```

8. Rollback – Migration-■ Undo ■■■■■■■

up() migration apply ■■■■■■■■. **down()** reverse ■■■■■■■■. Latest migration batch-■ undo ■■■■■:

```
php artisan migrate:rollback
```

Useful commands:

Command	Purpose
php artisan migrate:status	Migration run ■■■■■■■■■■■■■■ ■■■■■ ■■■■■■■
php artisan migrate:rollback --step=1	One migration step-■ rollback ■■■■■
php artisan migrate:reset	Run ■■ migrations-■ rollback ■■■■■
php artisan migrate:fresh	Tables drop ■■■■■■ migrations-■ fresh-■ run ■■■■■; existing data delete ■■■■■

9. Practical Example – Courses & Enrollments

Courses table:

```
$table->id();
$table->string('name')->unique();
$table->text('description')->nullable();
$table->decimal('price', 8, 2)->default(0);
$table->boolean('is_active')->default(true);
$table->timestamps();
```

Enrollments table:

```
$table->id();
$table->foreignId('student_id')->constrained('students')->restrictOnDelete();
$table->foreignId('course_id')->constrained('courses')->restrictOnDelete();
$table->string('status')->default('pending');
$table->unique(['student_id', 'course_id']);
$table->timestamps();
```

10. Migration Creation Flow

1. Migration create: **php artisan make:migration create_courses_table**
2. Migration code write ■■■■■■■■■.
3. **php artisan migrate** run ■■■■■■■■■.
4. Database-■■ table verify ■■■■■■■■■.
5. **php artisan migrate:status** ■■■■■ migration status check ■■■■■■■■■.

11. Already Existing Table-■ Alter ■■■■■■■

■■■ migration ■■■■■■■ run ■■■■■■■■■■■, ■■■■ ■■■■ migration file-■ edit ■■■■■■ ■■■■■■■■ migrate ■■■■■■■ normal workflow ■■■■. Existing table structure-■ change ■■■■■ ■■■■■ migration create ■■■■■■■ recommended.

```
php artisan make:migration add_code_to_courses_table --table=courses
```

```
public function up(): void
{
    Schema::table('courses', function (Blueprint $table) {
        $table->string('code')->unique();
    });
}
```

Important: Database-■■ manual-■■ ■■■■■■■ column add ■■■■■■■■■■■■■■■, ■■■ column-■ add ■■■■■■■■■ migration-■ blindly run ■■■■■■■■■■■■■■; duplicate-column error ■■■■■■■. Existing

schema ■■■■■■■ migration history-■ reconcile ■■■■■■ ■■■■■■■■■.

12. Manual DB Edit vs Migration

Situation	Recommended approach
New table / structure change	Migration
Existing table structure change	New migration
Normal data update	Application code / query; migration usually not needed
Production/shared DB structure change	Reviewed migration
Temporary local experiment	Manual DB edit possible, but be careful

13. Key Rule to Remember

Migration = Database Structure + Version Control

Manual DB edit local-■■ quick-■■ ■■■■■■■■■■■. ■■■■■■ Laravel project/team/production workflow-■■ migration use ■■■■■■■■■ schema changes Git-■■ track ■■■■■■; ■■■■■ developers ■■■■■■■■ deployment environments-■■■■■ ■■■■ change reproduce ■■■■■■ ■■■■■■■■■.

14. Day 9 Revision – One Minute

Concept	Remember
FK	One table record ■■■■■■■■ table record-■ reference ■■■■■■■■■
unique	Duplicate value/combination prevent ■■■■■■■■■
index	Search/query performance-■■■■■ ■■■■■■■
nullable	NULL value allow ■■■■■■■■■
default	Value ■■■■■■■■■■■■ default value
cascade	Parent delete → related child rows delete
restrict	Related rows ■■■■■■■■■■ parent delete prevent
up()	Apply change
down()	Reverse change